

Atomära händelser i praktiken: En genomgång av Unit of Work i ACME

Inom distribuerade system och AI-arkitekturer är den mest kritiska risken inte nödvändigtvis en systemkrasch, utan uppkomsten av partiella sanningar. Arkitekturen i ACME bygger på strikta **arkitektoniska invarianter** som säkerställer att systemet aldrig hamnar i ett inkonsistent mellanläge.

1. Introduktion: Problemet med partiella sanningar

Det grundläggande problemet som Unit of Work (UoW) löser i ACME-kontexten är risken för systemfel i glappet mellan att ett modellresultat erhålls och att dess effekter persistenslagras. Enligt ADR-0003 och ADR-0017 är "allt eller inget" fundamentalt för systemets integritet. **De tre största riskerna med partiella skrivningar:**

- **Inkonsistent tillstånd:** Att systemets olika delar (exempelvis långtidsminne och nuvarande status) hamnar i otakt, vilket korrumperar framtida beslut.
- **Förlorad historik:** Att en händelse utförs men spåret av *varför* den utfördes (evidensen) aldrig sparas, vilket omöjliggör auditering.
- **Integritetsförlust:** Den största risken; att systemet tvingas fatta beslut baserat på "spökdata" från misslyckade transaktioner. För att eliminera dessa risker har ACME implementerat en strikt koordination där alla sidoeffekter befordras atomärt.

2. Anatomien av ett Commit: Vad samordnas?

När en exekvering når sitt slut i ACME, koordineras fem kärnkomponenter i en transaktion (ADR-0006). Dessa samverkar för att skapa en sammanhängande bild av sanningen. | Effekttyp | Pedagogisk beskrivning || ----- | ----- || **Tillståndsändringar** | Uppdateringar av systemets nuvarande status (State), styrda av strikt revisionskontroll. || **Minnesbeslut** | Uppdateringar av långtidsminnet baserat på domänpolicyns tolkning av den senaste interaktionen. || **Domänhändelser** | Signaler för asynkron kommunikation (outbox), sparade i samma transaktion som tillståndet. || **Dokument** | Beständiga artefakter (exempelvis kapitel eller rapporter) som genererats under körningen. || **Terminal exekveringsprojektion** | En sammanfattning av körningen som tillåter externa observatörer att inspektera resultatet utan att läsa råa loggar. |

Dessa komponenter är inte bara samlade; de är tekniskt sammanflätade genom en central punkt som fungerar som systemets grindvakt.

3. ExecutionRepository: Den enda källan till sanning

I ACME-arkitekturen fungerar ExecutionRepository som navet genom vilket alla ändringar flödar. Vi tillämpar principen om **"One aggregate repository"** (ADR-0006). För att garantera transaktionell integritet förbjuder ACME oberoende portar för tillstånd eller minne. Om minnet kunde uppdateras via en separat port skulle vi riskera att minnet persistenslagras även om den övergripande transaktionen misslyckas. ExecutionRepository fungerar därför som systemets definitiva **Promotion Boundary**. "Atomär befordran av kandidat-till-kanonisk är en primär invariant." Detta innebär att all data existerar som en icke-publik "kandidat" fram till det ögonblick då hela paketet befordras till "kanonisk sanning".

4. Processen: Från kandidat till kanonisk sanning

Genomförandet av en exekvering följer en deterministisk sekvens som möjliggör återspelning:

1. **Förberedelse:** Skapande av kandidater i MemoryEngine och StateEngine. Dessa ligger initialt i arbetsminnet.
2. **Input-Bound Validering (ADR-0010):** ACME validerar **Contract Input** innan respospipelinen börjar bearbeta text. Detta säkerställer att vi inte försöker reparera ett AI-svar baserat på en felaktig begäran.
3. **Digest-beräkning (ADR-0006 & ADR-0012):** Algoritmen acme-operation-digest-1 beräknar ett unikt fingeravtryck av allt som ska sparas. Determinismen säkras genom **kanoniska sorteringsregler** (dokument sorteras efter nyckel, utvärderare efter identitet). Utan denna strikta ordning skulle innehållsidentiteten brytas vid återspelning.
4. **Commit (ADR-0013):** Den atomära skrivningen sker via SQLite med BEGIN IMMEDIATE. Eftersom SQLite är ett **single-writer system** används detta kommando för att förhindra deadlocks och säkerställa att transaktionen äger skrivlåset från start.

5. Integritet genom CAS och Digest

För att skydda mot korruption och samtidighetsproblem används tekniska mekanismer som gör att ACME kan **faila säkert i miljöer med flera skribenter** utan behov av tunga orkestreringslager:

- **Compare-and-Swap (CAS):** Systemet kontrollerar att versionen vi försöker uppdatera matchar den senaste kända sanningen.
- För tillstånd används CONFLICT_STATE_REVISION.
- För minne används CONFLICT_MEMORY_VERSION.
- **Identitet vs Innehåll (ADR-0004/0006):** Arkitekturen separerar dessa begrepp. **Operation Identity** (operationKey, härlett från requesten) skyddar operationens identitet, medan **Content Integrity** (acme-operation-digest-1) garanterar att innehållet inte divergerar. Detta säkerställer att samma logiska operation aldrig sparas med två olika resultat.

6. Hållbarhet och återhämtning: Durable Resume

Unit of Work möjliggör funktionen "Durable Resume" (ADR-0017), vilket innebär att vi kan återhämta oss från krascher utan extra kostnader för AI-anrop. En avgörande teknisk nyans är att en återupptagen körning **läser om kontexten** (loadContext) snarare än att återställa den lagrade "read set"-datan. Detta görs för att säkerställa att systemet beräknar mot det faktiska, aktuella tillståndet och förhindrar skrivningar baserade på gammal data (stale writes).!IMPORTANT **Viktig insikt:** En lyckad resume gör aldrig ett nytt anrop till AI-providern om ett svar redan finns registrerat. Om svarshistorik saknas (exempelvis vid hash-only policy) sätts exekveringen i det terminala läget RESUME_EVIDENCE_UNAVAILABLE.

7. Sammanfattning: Kraften i atomära händelser

ACME:s användning av Unit of Work handlar i slutändan om deterministisk tillit:

- **Determinism:** Kanoniska sorteringsregler och strikta algoritmer gör att identiska indata alltid resulterar i identiska commits.
- **Säkerhet:** Genom att använda CAS och BEGIN IMMEDIATE elimineras risken för halvfärdiga resultat och krockar mellan skribenter.
- **Effektivitet:** Koordination genom ExecutionRepository minimerar redundanta anrop och möjliggör hållbar återhämtning. I ACME är sanningen inte något som växer fram fragmentariskt; den etableras genom en befordran av kandidater till kanonisk sanning i ett enda, obrytbart ögonblick.